BSP Registry Tools

Streamlined Yocto & Isar BSP Management
for Embedded Linux Teams

From registry to built image — one command

```
pip install bsp-registry-tools
github.com/Advantech-EECC/bsp-registry
License:  Apache 2.0
```

Contents

1. Overview

BSP Registry Tools is a Python CLI and library for managing Board Support Packages (BSPs) for embedded Linux systems. It uses a YAML-based registry (schema v2.0) as the single source of truth, covering devices, Yocto/Isar releases, composable features, named presets, Docker container definitions, cloud deployment, and CI/CD integration.

- Python CLI tool (`bsp`) and importable Python API (`BspManager`, `V2Resolver`, . . .)
- YAML registry v2.0 — devices, releases, features, presets, environments
- Wraps KAS (`kas` / `kas-container`) for reproducible Yocto and Isar builds
- Zero-config first run — auto-clones the Advantech registry on first use
- Cloud artifact deploy / gather (Azure Blob Storage, AWS S3)
- HTTP server with REST + GraphQL APIs (FastAPI + Strawberry)
- Shell tab completions for all arguments

2. System Architecture

The diagram below shows how the BSP Registry Tools components interact with each other and with external systems. The `BspManager` API is the central hub connecting all user-facing interfaces (CLI, TUI/GUI explorer, HTTP server) to the underlying build infrastructure (KAS, container engines, cloud storage, LAVA test infrastructure).

2.1 Component Roles

Component	Role
bsp CLI	Primary user interface — build, list, export, deploy, gather, registry CRUD
bsp-explorer	TUI / GUI interface (interactive terminal explorer)
bsp server	HTTP REST + GraphQL server for automation and dashboards
BspManager API	Core Python library coordinating all operations
BSP Registry YAML	Single source of truth: devices, releases, features, presets
KAS Build System	Orchestrates Yocto BitBake or Isar builds from YAML config files
Container Engine	Docker / Podman provides isolated, reproducible build environments
Cloud Storage	Azure Blob / AWS S3 for artifact upload (`deploy`) and download (`gather`)
HIL Test / LAVA	Hardware-in-the-loop test automation via LAVA

3. Installation

Python 3.8 or later is required. A virtual environment is strongly recommended.

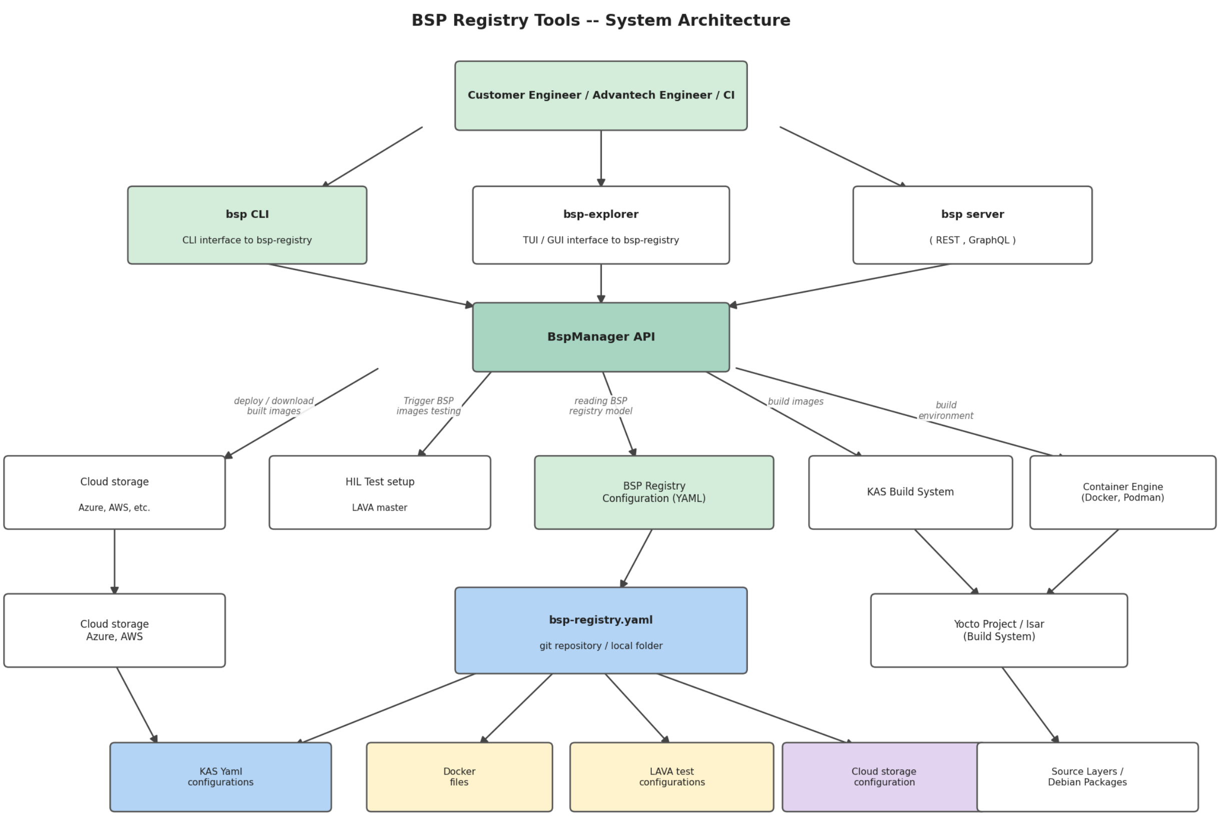

Figure 1: BSP Registry Tools — System Architecture

3.1 Core install

```
pip install bsp-registry-tools
```

3.2 Optional extras

Extra	Installs
`[azure]`	Azure Blob Storage upload / download
`[aws]`	AWS S3 upload / download
`[server]`	FastAPI + uvicorn + Strawberry GraphQL
`[completions]`	Shell tab completions (argcomplete)
`[dev]`	pytest, coverage, ruff, …

```
pip install "bsp-registry-tools[azure,completions]"
```

4. Supported Hardware

4.1 NXP i.MX Boards (Advantech Europe)

Board	SoC	Yocto Releases	Status
RSB-3720	i.MX8M Plus	scarthgap, styhead, walnascar, whinlatter	Stable
RSB-3720 4G	i.MX8M Plus	walnascar, whinlatter	Stable
RSB-3720 6G	i.MX8M Plus	walnascar, whinlatter	Stable
ROM-2620	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-2820	i.MX93	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-5720	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-5721	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
ROM-5722	i.MX8	scarthgap, styhead, walnascar, whinlatter	Stable
AOM-5521	i.MX95	scarthgap, walnascar	Stable

4.2 MediaTek & Qualcomm Boards

Board	SoC	Releases	Status
RSB-3810	MediaTek MT8395	scarthgap	Development
Genio 1200 EVK	MediaTek MT8395	scarthgap	Development
AOM-2721	Qualcomm QCS6490	scarthgap	Development
QCS6490 RB3 Gen2	Qualcomm QCS6490	scarthgap	Development

4.3 Emulated Targets (QEMU)

- `qemuarm64` — 64-bit ARM (AArch64)
- `qemuarm` — 32-bit ARM
- `qemux86` — x86 32-bit
- `qemux86-64` / `qemuamd64` — x86-64

5. Supported Releases

5.1 Yocto Releases

Slug	Yocto Version	LTS	Notes
kirkstone	4.0	LTS	poky, fsl-imx-xwayland
mickledore	4.2		poky, fsl-imx-xwayland
nanbield	4.3		poky
scarthgap	5.0	LTS	poky, fsl-imx-xwayland, mediatek, qualcomm, ros2
styhead	5.1		poky, fsl-imx-xwayland
walnascar	5.2		poky, fsl-imx-xwayland
whinlatter	5.3		poky, fsl-imx-xwayland
wrynose	6.0		poky (upcoming)

5.2 Isar (Debian-based) Releases

Slug	Distribution	Base Image
debian-trixie	Debian 13 (Trixie)	isar-debian-13 container
ubuntu-noble	Ubuntu 24.04 LTS	isar-debian-13 container
ubuntu-jammy	Ubuntu 22.04 LTS	isar-debian-13 container

6. Registry Schema v2

The registry uses a YAML file (default: `bsp-registry.yml`) with schema version 2.0. The file decomposes into independent, reusable sections:

Section	Purpose
`frameworks`	Build-system definitions (Yocto, Isar)
`distro`	Distribution definitions (Poky, fsl-imx-xwayland, ...)
`vendors`	Cross-release board-vendor KAS fragments
`devices`	Hardware board definitions (slug, vendor, SoC, KAS includes)
`releases`	Yocto / Isar release definitions with vendor overrides
`features`	Optional add-ons (OTA, secure-boot, ROS 2, hailo, ...)
`bsp`	Named presets = device + release(s) + features
`environments`	Container + variable bundles per build class
`containers`	Docker image definitions
`deploy`	Cloud artifact upload configuration (Azure / AWS)
`include`	Split large registries across multiple files

6.1 Device Definition Example

```yaml
registry:
  devices:
    - slug: rsb3720
      description: "Advantech RSB-3720 (i.MX8M Plus, 6GB)"
      vendor: advantech-europe
      soc_vendor: nxp
      architecture: arm64
      includes:
        - vendors/advantech-europe/nxp/machine/imx8/rsb3720.yml
```

6.2 Named Preset Example

```yaml
registry:
  bsp:
    - name: modular-bsp-rsb3720
      description: "Advantech RSB-3720 (i.MX8)"
      device: rsb3720
      releases: [scarthgap, styhead, walnascar, whinlatter]
      features: [systemd, security, virtualization, ipv6, usrmerge]
      build:
        path: build/modular-bsp-rsb3720
```

6.3 Vendor Override Example

Vendor overrides allow a single release slug (e.g. `scarthgap`) to apply different KAS fragments and distro settings per hardware vendor / SoC combination, with optional kernel sub-release pinning:

```yaml
releases:
  - slug: scarthgap
    distro: poky
    includes:
      - compilers/clang/clang.yml
      - yocto/releases/scarthgap.yml
    vendor_overrides:
      - vendor: advantech-europe
```

```
        soc_vendors:
          - vendor: nxp
            distro: fsl-imx-xwayland
            includes:
              - vendors/advantech-europe/nxp/modular-bsp-nxp.yml
            releases:
              - slug: imx-6.6.52-2.2.2
                includes:
                  - vendors/advantech-europe/nxp/imx-6.6.52-2.2.2-
                     scarthgap.yml
```

7. Feature System

Features are optional, composable add-ons that can be mixed into any BSP preset. They support `vendor_overrides` and `release_overrides` so the correct KAS fragments are injected automatically based on the target hardware and Yocto release.

Slug	Description
`systemd`	Enable systemd as the init system
`yocto-ssh`	Include SSH server in the image
`debug-tweaks`	Enable debug build tweaks (empty root password, ...)
`root-login`	Enable root login
`security`	Security hardening meta-layer
`secure-boot`	NXP HAB / AHAB cryptographic image signing
`virtualization`	KVM / container virtualisation support
`wayland`	Wayland display server support
`x11`	X11 display server support
`ipv6`	IPv6 networking
`udev`	udev device manager
`usrmerge`	/usr merge (FHS compliance)
`rauc`	RAUC Over-the-Air update framework
`swupdate`	SWUpdate OTA framework
`ostree`	OSTree atomic filesystem upgrades
`hailo`	Hailo-8 AI accelerator support
`ros2`	ROS 2 via ros2-humble-scarthgap release

8. OTA Update Support

Three OTA technologies are supported as first-class composable features. All major Advantech Europe NXP boards support all three methods.

Technology	Slug	Supported Boards	Releases
RAUC	`rauc`	RSB-3720, ROM-2620, ROM-2820, ROM-5720/21/22	scarthgap – whinlatter
SWUpdate	`swupdate`	RSB-3720, ROM-2620, ROM-2820, ROM-5720/21/22	scarthgap – whinlatter
OSTree	`ostree`	RSB-3720, ROM-2620, ROM-2820, ROM-5720/21/22	scarthgap – whinlatter

```
# Build RSB-3720 with RAUC OTA (Walnascar release)
bsp build modular-bsp-rauc-rsb3720 --release walnascar

# Build RSB-3720 with SWUpdate
bsp build modular-bsp-swupdate-rsb3720 --release scarthgap

# Build RSB-3720 with OSTree
bsp build modular-bsp-ostree-rsb3720 --release walnascar
```

9. Secure Boot

NXP-based Advantech boards support cryptographic boot-image signing via HAB (i.MX8 family) and AHAB (i.MX9 / i.MX95 family). The `secure-boot` feature uses the `ubuntu-22.04-csb` container which automatically mounts the NXP CST signing tool and keys from the host into the container.

SoC Family	Technology	Boards
i.MX8	HAB (High Assurance Boot)	RSB-3720, ROM-2620, ROM-5720, ROM-5721, ROM-5722
i.MX93	AHAB (Advanced High Assurance Boot)	ROM-2820
i.MX95	AHAB	AOM-5521

```
# Export signing key environment variables
export CST_TOOL_PATH=/opt/nxp/cst/
export KEYS_PATH=/path/to/srk-keys/

# Build ROM-5721 2G with Secure Boot (uses ubuntu-22.04-csb container)
bsp build modular-bsp-rom5721-2g-db5901-secureboot --release walnascar
```

10. Containers & Environments

Container definitions bundle a Dockerfile, a Docker image name, build args, and optional volume mounts. Named environments combine a container with build-system environment variables.

Container	Base OS	KAS	Use Case
ubuntu-20.04	Ubuntu 20.04	4.7	Kirkstone / Mickledore legacy builds
ubuntu-22.04	Ubuntu 22.04	5.2	Default Yocto builds
ubuntu-22.04-csb	Ubuntu 22.04	5.2	Secure Boot builds (key mounts)
ubuntu-24.04	Ubuntu 24.04	5.2	Latest Yocto builds
debian-12	Debian 12	5.2	Hardened / alternative builds
debian-13	Debian 13	5.2	Debian-based Yocto builds
isar-debian-13	Debian 13	5.2	Isar privileged builds

11. CLI Reference

Command	Description
bsp list	List all named BSP presets
bsp list devices	List all device slugs
bsp list releases	List all release slugs
bsp list features	List all feature slugs
bsp containers	List container definitions
bsp build <preset>	Build a named preset
bsp build <preset> -release <r>	Build with a specific release
bsp build <preset> -checkout	Validate config (no actual build)
bsp build <preset> -deploy	Build and deploy to cloud storage
bsp build <preset> -clean	Clean build dir before building
bsp shell <preset>	Open shell in build container
bsp export <preset>	Export resolved KAS config
bsp deploy <preset>	Upload artifacts to cloud storage
bsp gather <preset>	Download artifacts from cloud
bsp registry init	Scaffold a new registry YAML file
bsp registry validate	Validate registry schema
bsp registry diff <f1> <f2>	Diff between two registry files
bsp registry add device ...	Add a new device entry
bsp registry add release ...	Add a new release entry
bsp remotes add <name> <url>	Save a named remote registry URL
bsp remotes show	List all saved remotes
bsp server	Start REST + GraphQL HTTP server
bsp completions bash|zsh|fish	Print shell completion script

11.1 Global CLI Flags

Flag	Description
`-registry <path>`	Use an explicit local registry file
`-local`	Use `./bsp-registry.yml`, no network
`-remote <url>`	Remote registry URL or saved remote name
`-branch <b>`	Branch to checkout when using `-remote`
`-no-update`	Skip git-pull of the remote registry

11.2 Quick Start

```
# First run: clones the Advantech registry automatically
bsp list

# Build Advantech RSB-3720 with the default release
bsp build modular-bsp-rsb3720

# Build with a specific Yocto release
bsp build modular-bsp-rsb3720 --release walnascar

# Validate config quickly (no full build)
bsp build modular-bsp-rsb3720 --checkout

# Use the Advantech development branch explicitly
bsp --remote https://github.com/Advantech-EECC/bsp-registry.
   git@development list
```

12. Python API

All CLI operations are backed by a public Python API. The key entry point is `BspManager`, which coordinates registry loading, preset resolution, builds, deployment, and artifact gathering.

```
from bsp import BspManager, RegistryFetcher, V2Resolver

# 1. Fetch / update the remote registry
fetcher = RegistryFetcher()
registry_path = fetcher.fetch_registry(
    repo_url="https://github.com/Advantech-EECC/bsp-registry.git",
    branch="development",
    update=True,
)

# 2. Load registry and inspect contents
manager = BspManager(str(registry_path))
manager.initialize()
for dev in manager.model.registry.devices:
    print(dev.slug, dev.description)

# 3. Resolve a preset into a full build configuration
resolver = V2Resolver(manager.model)
config = resolver.resolve(
    device_slug="rsb3720",
```

```
    release_slug="walnascar",
    feature_slugs=["systemd", "security", "rauc"],
)
for f in config.kas_files:
    print(f)
```

12.1 Multi-Registry Mode

```
from bsp import BspManager

manager = BspManager(
    config_paths=[
        ("advantech",    "/path/to/advantech-registry.yaml"),
        ("my-additions", "/path/to/my-registry.yaml"),
    ]
)
manager.initialize()
# Disambiguate with registry:preset syntax
config = manager.resolve("advantech:modular-bsp-rsb3720")
```

13. HTTP Server (REST + GraphQL)

```
pip install "bsp-registry-tools[server]"
bsp server --port 8080
```

13.1 REST API Endpoints

Method	Path	Description
GET	`/api/v1/devices`	List all devices
GET	`/api/v1/releases`	List all releases
GET	`/api/v1/features`	List all features
GET	`/api/v1/bsp`	List all named presets
POST	`/api/v1/bsp/{name}/build`	Trigger a build
POST	`/graphql`	GraphQL endpoint

```
# GraphQL query example
curl -X POST http://localhost:8080/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query": "{ releases { slug description yoctoVersion } }"}'
```

14. CI/CD Integration

```
# .github/workflows/build.yml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4

      - name: Install bsp-registry-tools
        run: pip install "bsp-registry-tools[azure]"

      - name: Validate registry
        run: bsp --local registry validate

      - name: Fast checkout gate (no full build)
        run: bsp --no-update build modular-bsp-rsb3720 --checkout

      - name: Build and deploy artifacts
        env:
          AZURE_STORAGE_ACCOUNT_URL: ${{ secrets.
             AZURE_STORAGE_ACCOUNT_URL }}
        run: bsp --no-update build modular-bsp-rsb3720 --release
           walnascar --deploy
```

15. Extending BSPs with Custom Yocto Layers

A BSP registry KAS configuration can be included in your own project, allowing you to add custom Yocto layers on top of an existing BSP without modifying the registry itself.

```
# my-custom-kas.yaml
header:
  version: 19
  includes:
    - repo: bsp-registry
      file: modular-bsp-rsb3720-walnascar.yaml   # from `bsp export`

repos:
  bsp-registry:
    url: "https://github.com/Advantech-EECC/bsp-registry"
    branch: "development"
    layers: { .: "disabled" }

  meta-custom:
    layers:
      meta-custom:    # your custom Yocto meta-layer
```

```
# meta-custom/meta-custom/recipes-core/imx-image-%.bbappend
CORE_IMAGE_EXTRA_INSTALL += "mpv"
LICENSE_FLAGS_ACCEPTED += "commercial"

# Build with your extension
kas build my-custom-kas.yaml
```

16. Summary

Feature	Benefit
YAML registry v2	Single source of truth for 30+ boards × 8+ releases
Auto remote fetch	Zero-config first run; teams share the Advantech registry
Two build systems	Yocto/BitBake AND Isar/Debian in one registry
KAS integration	Reproducible builds for every device × release combo
Named environments	Correct container per build class (secure-boot, Isar, . . .)
Feature system	Composable add-ons: OTA, secure-boot, ROS 2, Hailo
OTA support	RAUC, SWUpdate, OSTree — all first-class features
Secure Boot	NXP HAB / AHAB signing built into the registry
Cloud deploy / gather	Full artifact lifecycle: build → upload → download
HTTP server	REST + GraphQL for dashboards and automation
Tab completions	Fast CLI use with context-aware completions
Python API	Integrate into custom tools and CI scripts

Registry: `https://github.com/Advantech-EECC/bsp-registry` *(development branch)*